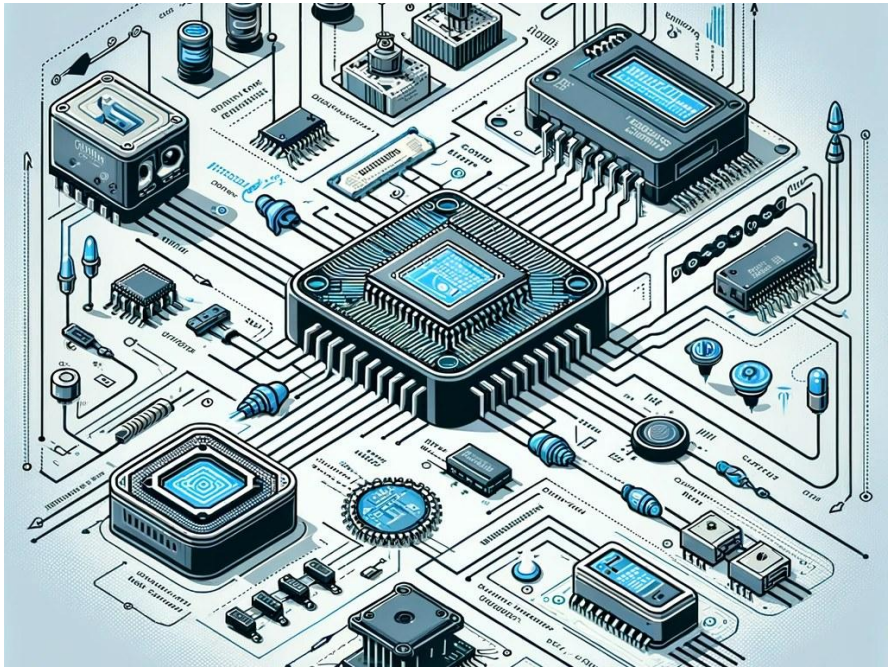
FALL 2026 | MEE4033

Mechatronics

Mechatronics (MEE4033) · Department of Mechanical Engineering

Lecture 10

Digital System Implementation: Interrupts



Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C, Chapter 11

Sogang University · Department of Mechanical Engineering · Prof. Seokhwan Jeong

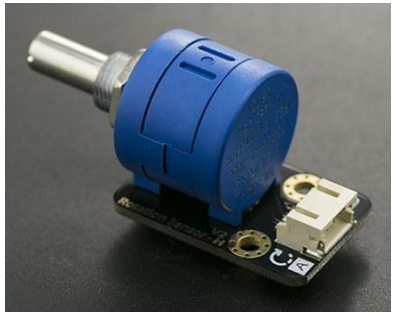
Digital Control System Implementation

- How to implement the discrete-time sampling?
- To implement the digital control system, we need a function providing a **constant sample period, T**
- This can be implemented by the ***interrupt*** of the MCU



Reading the Sensor Signal: Analog Method

- How to obtain/calculate the sensor signal (state)?
  - **Analog method** + ADC (analog-to-digital converter)



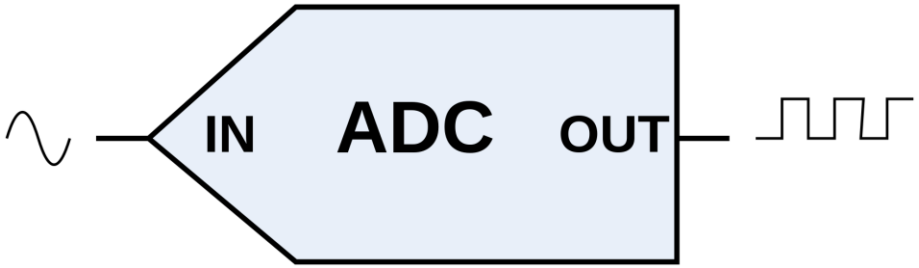
Potentiometer



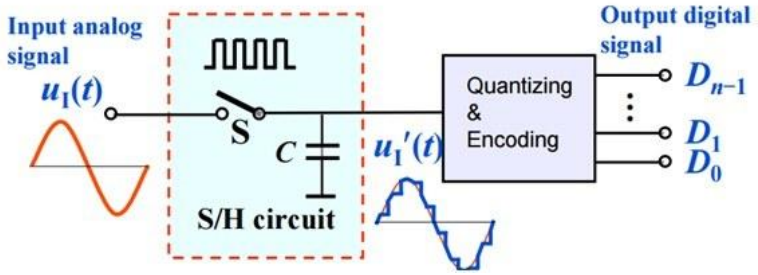
Force sensing resistor



Light sensor



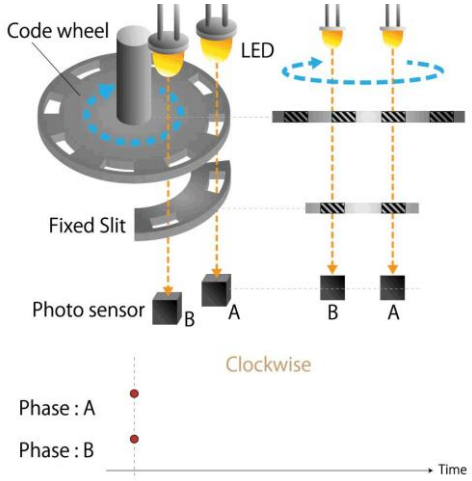
en.wikipedia.org: Analog-to-digital converter (ADC symbol)



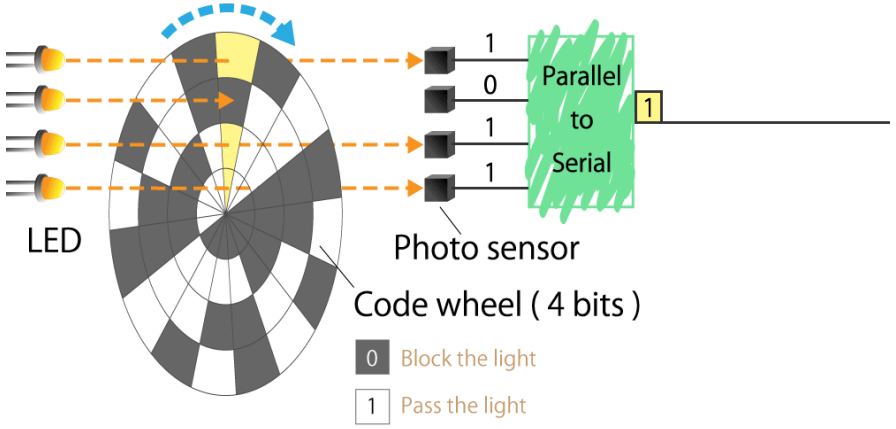
microcontrollerslab.com: analog-to-digital converter working

Reading the Sensor Signal: Digital Method

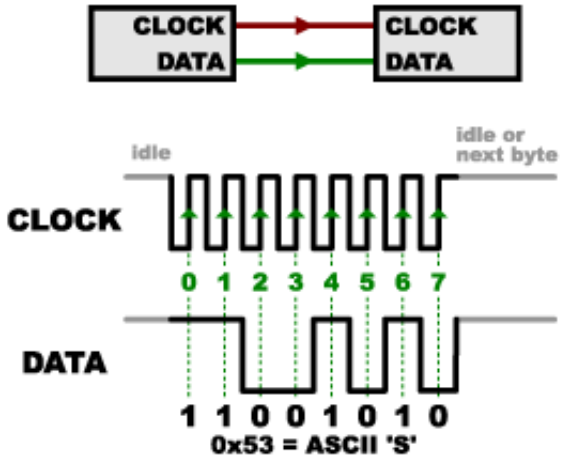
- How to obtain/calculate the sensor signal (state)?
  - **Digital method** + digital input or communication
  - This can also be implemented by the ***interrupt*** of the MCU



Hall sensor-based encoder (incremental method)



Hall sensor-based encoder (absolute method)



Serial communication (clock + data)

youtube.com/watch?v=BFZceDq7YXg (encoder animation)

Introduction to Interrupts

- An **interrupt** leverages a combination of software and hardware to force the processor to stop its current activity and begin to execute a particular piece of code called an ***interrupt service routine (ISR)***
- An **ISR** responds to a **specific event** generated by either hardware or software
- When an ISR completes, the processor automatically **resumes** the activity that had been halted
- Interrupts allow a processor to perform **multiple tasks simultaneously**
- Applications: pushing a button, timer, program shutdown, receiving a message in a communication port

Polling vs. Interrupt



- **Polling scheme:** the processor **continually queries** the I/O devices to check whether a specific event has happened and handles the event. The **latency** of detecting the event is determined by the polling period.
- **Interrupt scheme:** the processor provides a hardware mechanism that allows an internal or external device to generate a signal to **immediately inform the processor of events** that have occurred.

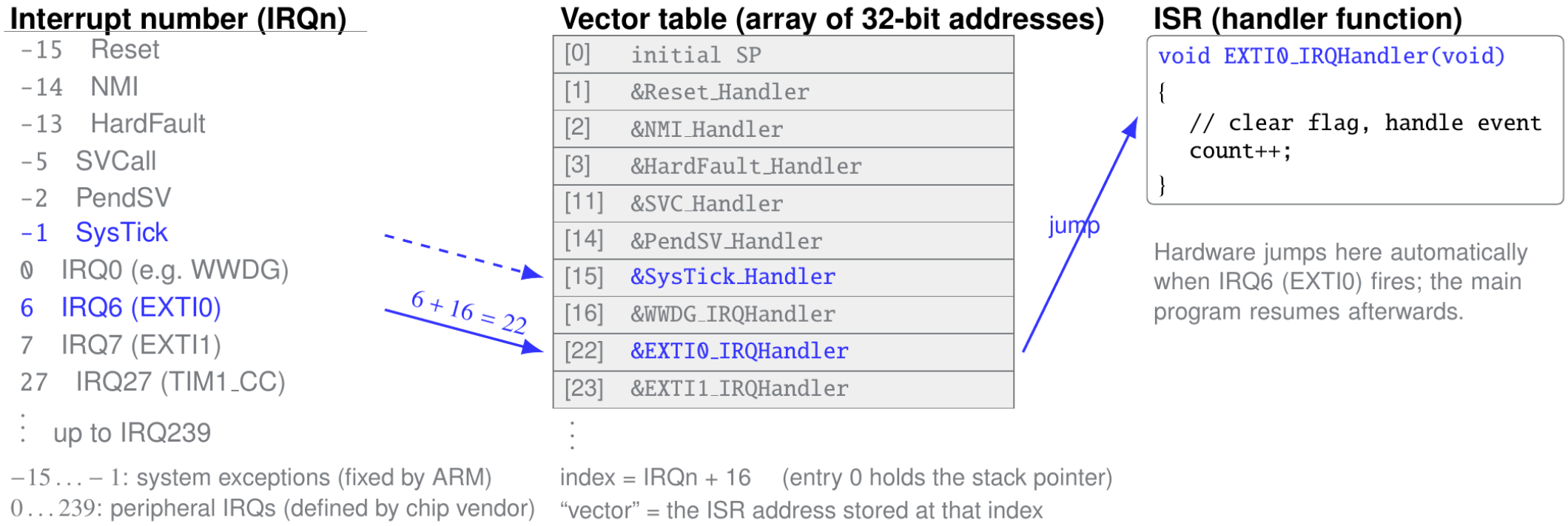
→ **Polling is much less efficient than using interrupts**

Interrupt Numbers

- For ARM Cortex-M processors (e.g., Arduino Due)
- Support up to 256 types of interrupts
- Interrupt numbers are defined by ARM and chip manufacturers
  - Each interrupt is identified by a unique number (e.g., −15 to 240)
- First 16 interrupts: **system interrupts** (system exceptions)
  - Supervisor call interrupts, system timer interrupts, fault-handling interrupts, etc.
- Remaining 240 interrupts: **peripheral interrupts** (**external interrupts**, non-system exceptions)
  - ADC interrupts, USB interrupts, serial communication interrupts (SPI, I^2C, USART)

Interrupt Number, Vector Table, and ISR

- **Interrupt number** (IRQn): an index. **Vector**: the ISR address stored at that index. **Vector table**: the array of those addresses



  - Cortex-M: table index = IRQn + 16 (16 system slots first). AVR (Table 11-1): the vector number itself is the index, starting at 1

Interrupt Service Routines

- An **interrupt service routine (ISR)**, also called **interrupt handler**, is a special subroutine that is invoked automatically in response to an interrupt; each ISR is associated with one type of interrupt
- E.g., ***Reset_Handler*** runs when the processor is ***reset or powered up*** and calls ***main()***; a ***timer ISR*** runs once every sample period

Cortex-M (e.g., Arduino Due): CMSIS style

```
// Runs first after power-up or reset
void Reset_Handler(void)
{
    SystemInit();       // clocks, memory
    main();             // never returns
}

// Runs every SysTick period (sample time T)
void SysTick_Handler(void)
{
    x = readSensor();  // ADC / encoder
    u = controller(x); // control law
    writeActuator(u);  // PWM
}
```

AVR (Arduino Uno): avr-libc style

```
// Table 11-1 vector 12: Timer1 COMPA
// runs when TCNT1 reaches OCR1A (CTC mode)
ISR(TIMER1_COMPA_vect)
{
    x = readSensor();
    u = controller(x);
    writeActuator(u);
}

// Table 11-1 vector 2: INT0 (pin D2)
ISR(INT0_vect)
{
    count++;        // e.g., encoder edge
}
```

The handler name is fixed by the startup code (its address sits in the vector table); the function takes no arguments and returns nothing. Keep it short: do the time-critical work, set a flag, and leave the rest to the main loop.

Interrupt Service Routines: Timer ISR Example

- ***Timer_ISR*** is executed at every sampling time interval

```
////////////////////////////////////////BLE setting END////////////////////////////////////
//  Serial.begin(115200);
  hrpwm.initperiod_all(200);  //switch on for all pwm pins PWM duty 200 um frequency 5kHz
//  hrpwm.initperiod(3, 500);  //switch on for all pwm pins PWM duty 200 um frequency 5kHz
//  hrpwm.initperiod(4, 500);  //switch on for all pwm pins PWM duty 200 um frequency 5kHz
//  hrpwm.initperiod(5, 500);  //switch on for all pwm pins PWM duty 200 um frequency 5kHz
  startTimerInt_period(TC1, 0, TC3_IRQn, 2000, 1); //TC1 channel 0, IRQ number, 2000us, priority 1


  // initialize the pins as an Output/Input

  pinMode(led, OUTPUT);
  pinMode(Encoder_1_A, INPUT_PULLUP);
  pinMode(Encoder_1_B, INPUT_PULLUP);
  pinMode(Encoder_2_A, INPUT_PULLUP);
```

  - setup(): startTimerInt_period(TC1, 0, TC1_IRQn, 2000, 1) registers the timer channel, the sampling period and the priority

```
void TC3_Handler()
{
  TC_GetStatus(TC1, 0);  // Needed for proper interrupt operation

  //Time1=micros();
  //Time2=micros();

  for(n=0;n<6;n++){

  Rot[n] = (float)Enc_RAM[n]/29.86/(2.0*14.0);
  Err[n] = Ref[n]-Rot[n];

  P_u[n] = 5*Err[n];
  if(abs(u_p[n])<1){
    I_u[n] = 0.004002*Err_p[n]+I_u_p[n];
    }
  D_u[n] = 11.66*Err[n]-11.66*Err_p[n]+0.5335*D_u_p[n];

  u[n]=P_u[n]+I_u[n]+D_u[n];

  Err_pp[n] = Err_p[n];
  Err_p[n] = Err[n];

  u_pp[n] = u_p[n];
  u_p[n] = u[n];

  D_u_p[n] = D_u[n];

  I_u_p[n] = I_u[n];


  //PWM & Direction//
  if(u[n]>=0) digitalWriteDirect(M_Dir[n], HIGH);
  else digitalWriteDirect(M_Dir[n], LOW);   //Motor Diriection

  duty[n] = abs((float)u[n]);  //duty range  0 ~ 0.99
  hrpwm.dutyset(PWM_num[n],duty[n]);
  }


}
```

Interrupt Numbers: ATmega328P (Arduino Uno)

ATmega328P (Arduino Uno)

- Table 11-1 of the datasheet: reset and interrupt vectors

Table 11-1. Reset and Interrupt Vectors in ATmega328P

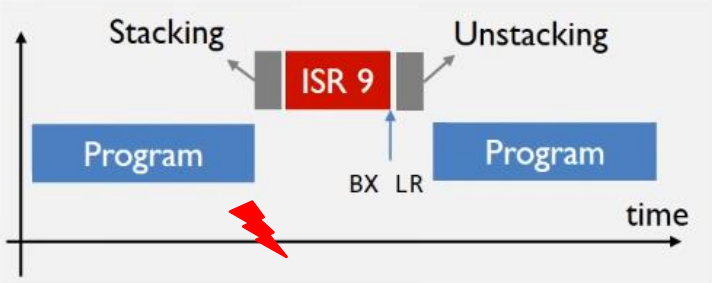
Vector No.	Program Address	Source	Interrupt Definition
1	0x0000	RESET	External pin, power-on reset, brown-out reset and watchdog system reset
2	0x002	INT0	External interrupt request 0
3	0x0004	INT1	External interrupt request 1
4	0x0006	PCINT0	Pin change interrupt request 0
5	0x0008	PCINT1	Pin change interrupt request 1
6	0x000A	PCINT2	Pin change interrupt request 2
7	0x000C	WDT	Watchdog time-out interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 compare match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 compare match B
10	0x0012	TIMER2 OVF	Timer/Counter2 overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 capture event
12	0x0016	TIMER1 COMPA	Timer/Counter1 compare match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 compare match B
14	0x001A	TIMER1 OVF	Timer/Counter1 overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 compare match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 compare match B
17	0x0020	TIMER0 OVF	Timer/Counter0 overflow
18	0x0022	SPI, STC	SPI serial transfer complete
19	0x0024	USART, RX	USART Rx complete
20	0x0026	USART, UDRE	USART, data register empty
21	0x0028	USART, TX	USART, Tx complete
22	0x002A	ADC	ADC conversion complete
23	0x002C	EE READY	EEPROM ready
24	0x002E	ANALOG COMP	Analog comparator
25	0x0030	TWI	2-wire serial interface
26	0x0032	SPM READY	Store program memory ready

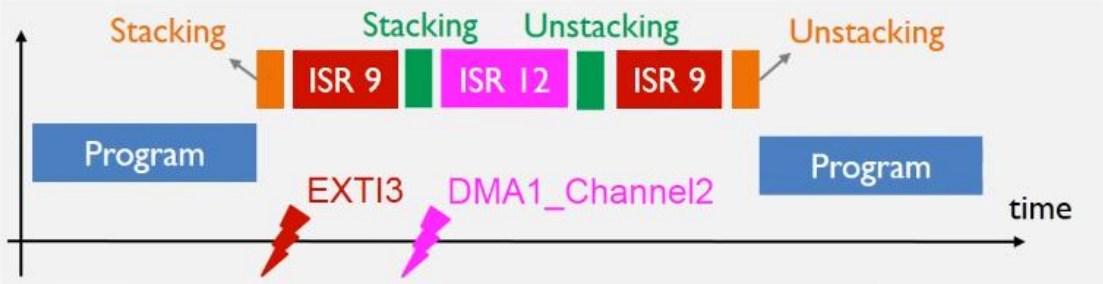
ww1.microchip.com: ATmega328P datasheet (Atmel-7810)

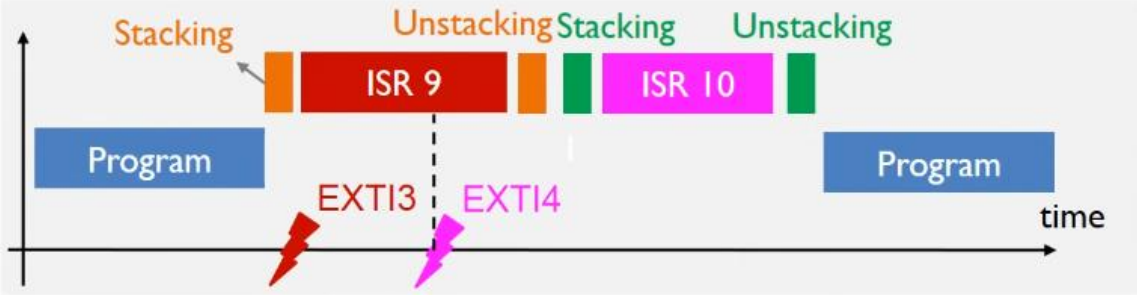
Interrupt Priority

- **Nested vectored interrupt controller (NVIC)** allows applications to enable/disable specific interrupts and set their **priority**
- The processor serves all interrupts based on their priority levels
- The processor **stops the currently running** interrupt handler if a new interrupt with a **higher priority occurs**; the processor **resumes** the low-priority handler when the handler of the new interrupt **completes**
  - A higher value of the interrupt priority number = a lower priority
  - Interrupts with the same priority are served in the order of their vector position
  - Reset_Handler() has the top priority

Interrupt Priority: Nesting Examples







youtube.com/watch?v=uFBNf7F3l60 (interrupt nesting animation)

Priority: ISR 9 < ISR 12

- ISR 12 pre-empts ISR 9; ISR 9 resumes after ISR 12 returns

Priority: ISR 9 > ISR 10

- ISR 10 waits until ISR 9 has finished

Timer Interrupt

- The system tick timer (**SysTick**) is a counter that produces a fixed time quantum
- Software uses **SysTick** to create time delays or to generate **periodic interrupts** to execute a task repeatedly
- This is the mechanism that gives the digital controller its **constant sample period T**

Timer Interrupt: SysTick Period Example

Example of calculating the system timer interval

Figure 11-11 shows an example of how to calculate the time interval between two consecutive SysTick interrupts. In this example, `SysTick_LOAD` is 6. If the processor clock is 1 MHz and the SysTick counter takes it as the input clock, then we can calculate the SysTick interrupt period as follows:

$$SysTick\ Interrupt\ Period = (1 + SysTick_LOAD) \times \frac{1}{SysTick\ Counter\ Clock\ Frequency}$$

$$= (1 + 6) \times \frac{1}{1MHz}$$

$$= 7\mu s$$

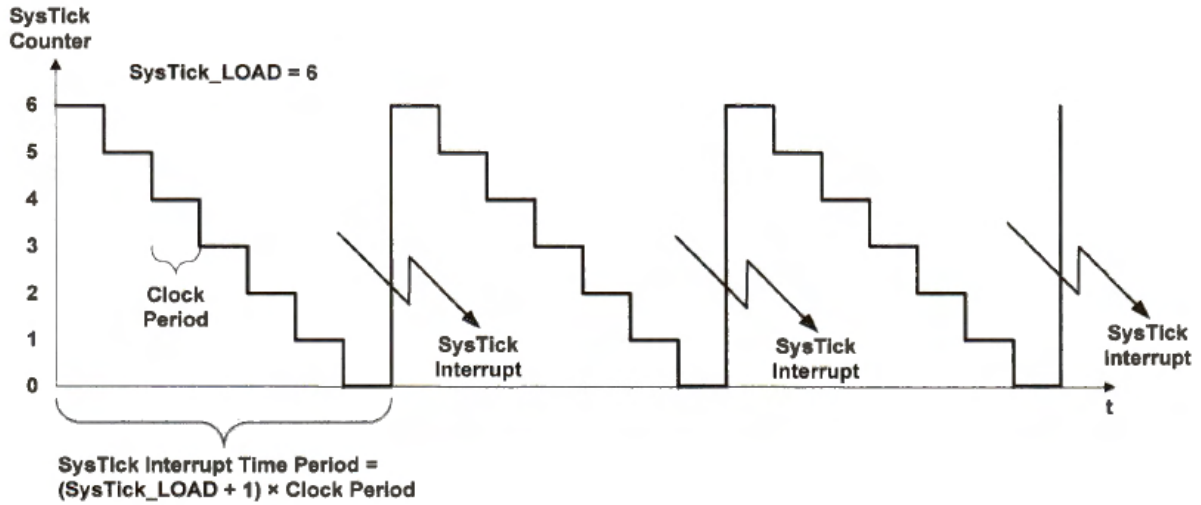
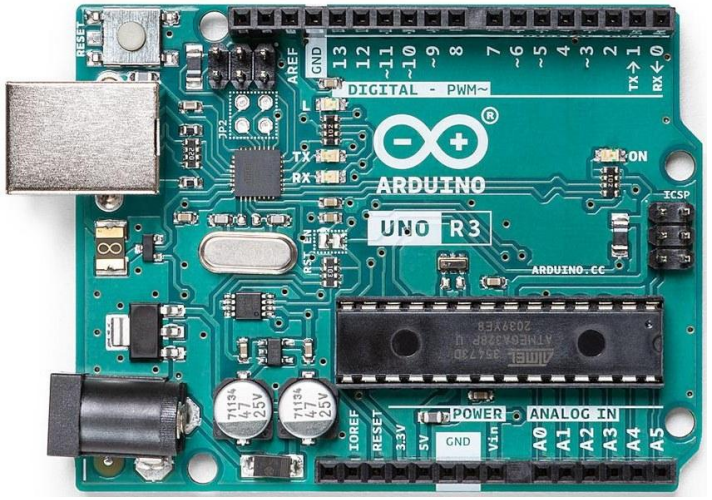


Figure 11-11. Example of SysTick interrupt when `SysTick_LOAD` is 6.

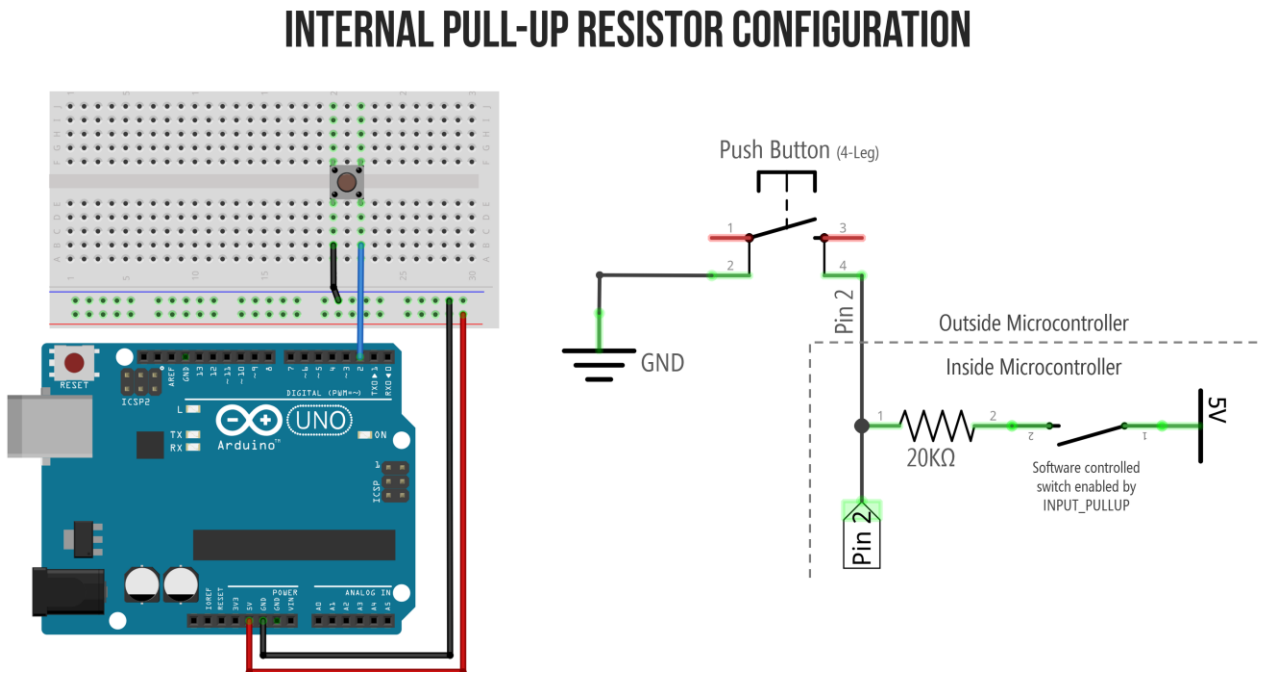


– Arduino Uno: Timer0/1/2 play the same role

External Interrupt

- External interrupts are interrupts **triggered by peripherals or devices external** to the MCU
- Push buttons, keypads, encoders (i.e., digital inputs)
- The external interrupt has a voltage monitor module with a programmable edge detector (rising edge, falling edge, both edges) → it triggers the ISR (e.g., EXTI3_IRQHandler())
- Named **EXTI0, EXTI1**, ... (or **INT0, INT1**, ...)
- Each of these interrupts is **associated with one specific GPIO** (general-purpose input/output) pin
- The datasheet of the MCU provides the interrupt mapping to the GPIO pins

External Interrupt: Push Button with Internal Pull-up



pinMode()

docs.arduino.cc: pinMode()

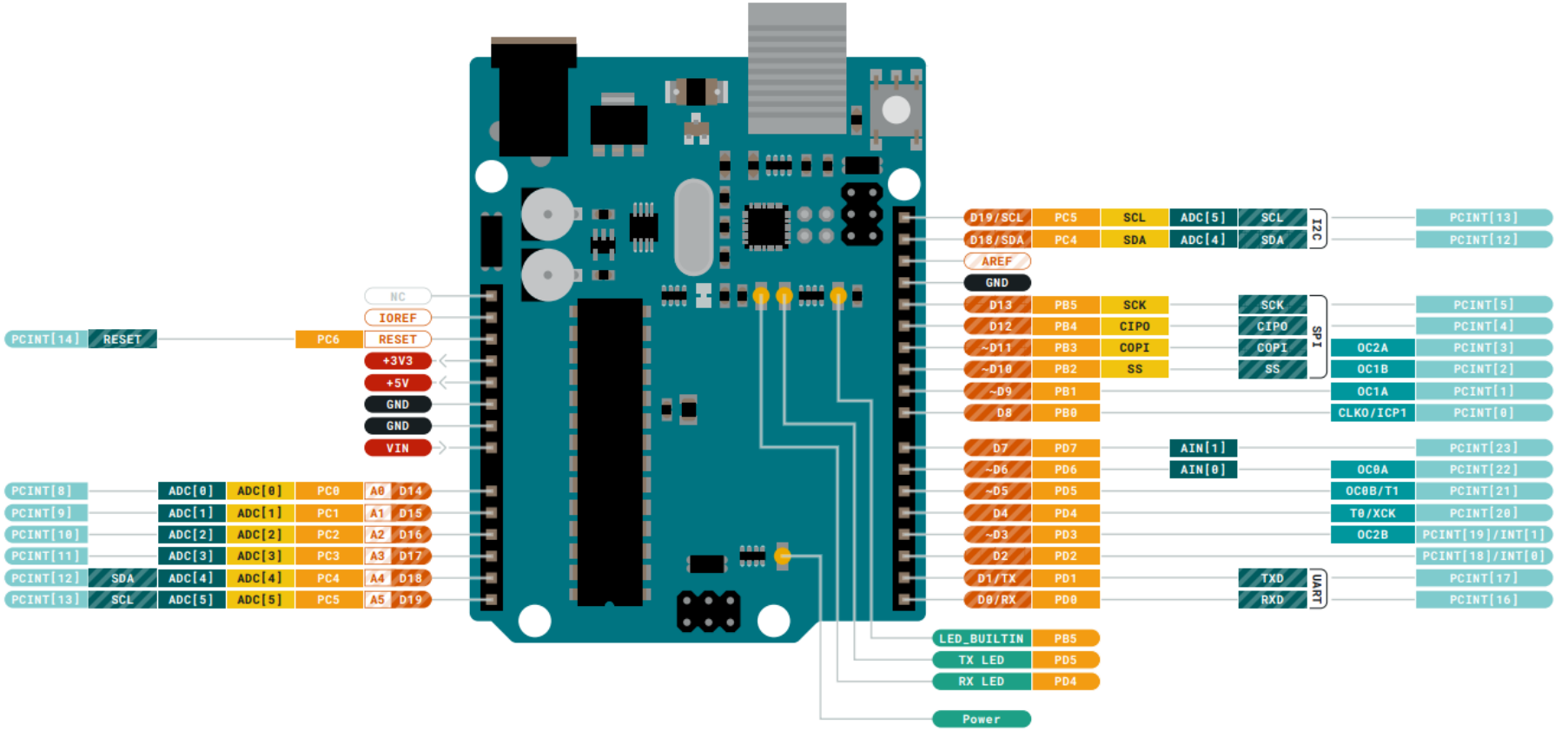
attachInterrupt()

docs.arduino.cc: attachInterrupt()

```
pinMode(2, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(2), isr, FALLING);
```

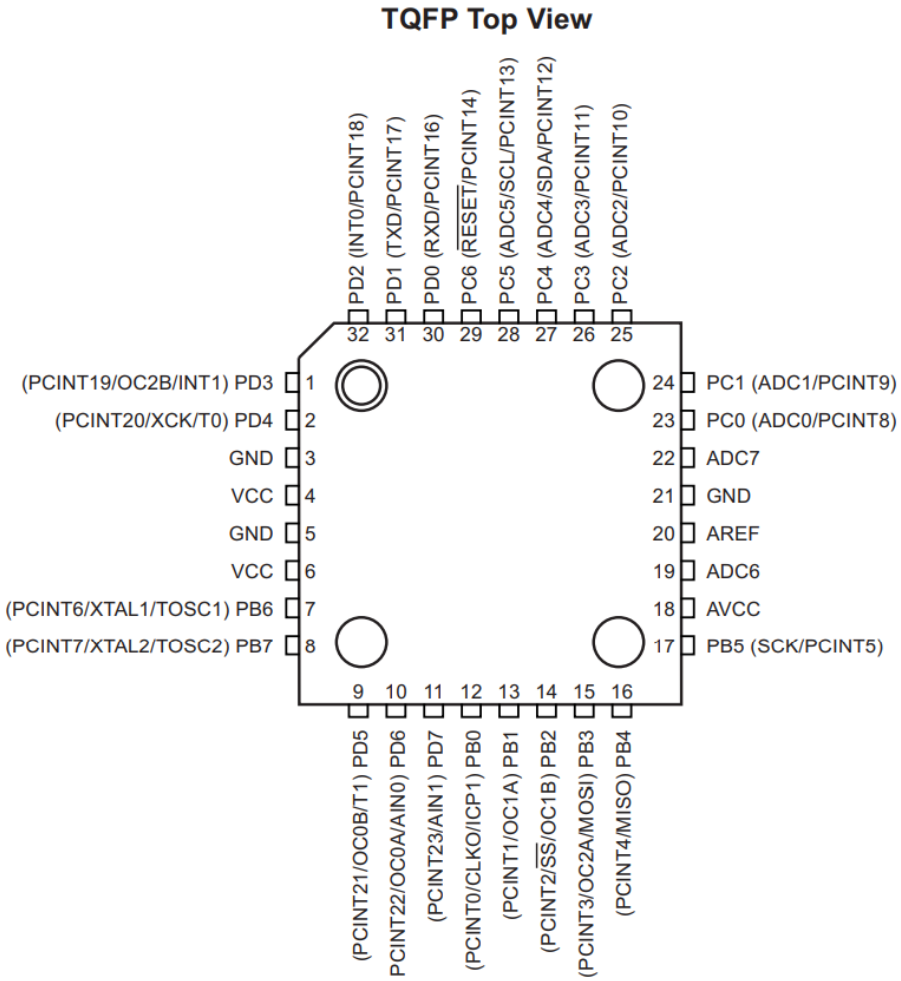
makeabilitylab.github.io: physcomp/arduino/buttons

External Interrupt: Arduino Uno Pinout



[store.arduino.cc: Arduino Uno Rev3 (pinout diagram)]

External Interrupt: ATmega328P Pin Mapping



[ww1.microchip.com: ATmega328P datasheet (Atmel-7810)]

External Interrupt: Port B Pins Alternate Functions (Table 13-3)

Table 13-3. Port B Pins Alternate Functions

Port Pin	Alternate Functions
PB7	XTAL2 (chip clock oscillator pin 2) TOSC2 (timer oscillator pin 2) PCINT7 (pin change interrupt 7)
PB6	XTAL1 (chip clock oscillator pin 1 or external clock input) TOSC1 (timer oscillator pin 1) PCINT6 (pin change interrupt 6)
PB5	SCK (SPI bus master clock input) PCINT5 (pin change interrupt 5)
PB4	MISO (SPI bus master input/slave output) PCINT4 (pin change interrupt 4)
PB3	MOSI (SPI bus master output/slave input) OC2A (Timer/Counter2 output compare match A output) PCINT3 (pin change interrupt 3)
PB2	$\overline{SS}$ (SPI bus master slave select) OC1B (Timer/Counter1 output compare match B output) PCINT2 (pin change interrupt 2)
PB1	OC1A (Timer/Counter1 output compare match A output) PCINT1 (pin change interrupt 1)
PB0	ICP1 (Timer/Counter1 input capture input) CLKO (divided system clock output) PCINT0 (pin change interrupt 0)

Source: ATmega328P datasheet (Atmel-7810)

External Interrupt: Port C Pins Alternate Functions (Table 13-6)

Table 13-6. Port C Pins Alternate Functions

Port Pin	Alternate Function
PC6	$\overline{\text{RESET}}$ (reset pin) PCINT14 (pin change interrupt 14)
PC5	ADC5 (ADC input channel 5) SCL (2-wire serial bus clock line) PCINT13 (pin change interrupt 13)
PC4	ADC4 (ADC input channel 4) SDA (2-wire serial bus data input/output line) PCINT12 (pin change interrupt 12)
PC3	ADC3 (ADC input channel 3) PCINT11 (pin change interrupt 11)
PC2	ADC2 (ADC input channel 2) PCINT10 (pin change interrupt 10)
PC1	ADC1 (ADC input channel 1) PCINT9 (pin change interrupt 9)
PC0	ADC0 (ADC input channel 0) PCINT8 (pin change interrupt 8)

Source: ATmega328P datasheet (Atmel-7810)

External Interrupt: Port D Pins Alternate Functions (Table 13-9)

Table 13-9. Port D Pins Alternate Functions

Port Pin	Alternate Function
PD7	AIN1 (analog comparator negative input) PCINT23 (Pin Change Interrupt 23)
PD6	AIN0 (analog comparator positive input) OC0A (Timer/Counter0 output compare match A output) PCINT22 (pin change interrupt 22)
PD5	T1 (Timer/Counter 1 external counter input) OC0B (Timer/Counter0 output compare match B output) PCINT21 (pin change interrupt 21)
PD4	XCK (USART external clock input/output) T0 (Timer/Counter 0 external counter input) PCINT20 (pin change interrupt 20)
PD3	INT1 (external interrupt 1 input) OC2B (Timer/Counter2 output compare match B output) PCINT19 (pin change interrupt 19)
PD2	INT0 (external interrupt 0 input) PCINT18 (pin change interrupt 18)
PD1	TXD (USART output pin) PCINT17 (pin change interrupt 17)
PD0	RXD (USART input pin) PCINT16 (pin change interrupt 16)

Source: ATmega328P datasheet (Atmel-7810)